

SOFTWARE REQUIREMENTS SPECIFICATION

AccessLearn

Accessible Multimodal Learning Platform

Converting educational material into audio, captions, video recommendation, Quiz module and Insights.

Category : AI Development | Difficulty: Hard

Team Name : VAAT MATRIX

Team Members : Vinotha S, Thaisree S, Anushya S, Anusiya R

1. Introduction

1.1 Purpose

This document defines the requirements, scope, and development plan for AccessLearn, an AI-driven platform that converts educational material into accessible formats. It serves as a reference for the design and development team throughout the build.

1.2 Problem Statement

Most learning content is created for a single type of learner, leaving others behind. Visually impaired learners cannot read dense slides or scanned PDFs. Hearing-impaired learners have no captions or on lecture videos. Learners with reading difficulties struggle with dense academic text, and time-constrained learners have no easy way to consume material on the go. AccessLearn addresses this by converting a single uploaded document into multiple accessible formats automatically.

1.3 Scope

The platform accepts a document, presentation, or lecture recording as input and produces four accessible outputs: simplified easy-read text, narrated audio, synced captions for key terms. The initial build focuses on PDF input, with other formats planned as extensions.

2. Product Description

AccessLearn is a web-based platform where a user uploads a single piece of learning material and receives it back in four parallel accessible formats through one dashboard. The system separates content extraction from format generation, so new output formats can be added without changing how material is ingested. The goal is to make accessibility a default feature of course content, not an afterthought handled manually by institutions.

3. Target Audience

AccessLearn is designed for individual learners with specific accessibility needs, as well as the institutions that serve them.

Audience Segment	Accessibility Need
Learners with Visual Impairment	Cannot read dense slides, scanned PDFs, or printed text without assistive support.
Learners with Hearing Impairment	Need captioned content for lecture videos that currently offer neither.
Learners with Reading Difficulty	Struggle to process dense academic text and benefit from simplified, easy-read rewrites.
Time-Constrained Learners	Need audio versions of material to study while commuting or multitasking.
Educational Institutions	Colleges and training providers seeking to make course material accessible by default.

4. System Pipeline

Uploaded material passes through one extraction step and four parallel processing services before reaching a single output dashboard, keeping the pipeline simple to build and extend.

5. User Workflow

The end-to-end experience is kept to five straightforward steps so a first-time user needs no onboarding.

6. Development Phases

The build is organized into a total of 6 phases, moving from planning through deployment. Each phase has a clear focus area and deliverable so progress can be tracked.

Phase	Focus Area	What To Do	Key Deliverable
Phase 1	Requirement Analysis & Planning	Gather accessibility requirements from target users; define supported input formats (PDF, PPT, video); finalize scope and success criteria.	Requirement document, project plan
Phase 2	System & UX Design	Design system architecture and data flow; design the upload and dashboard interface; define accessible-format standards (captions, audio, easy-read text).	Architecture diagram, UI wireframes
Phase 3	Content Extraction Module	Build the module that extracts raw text, images, and structure from uploaded documents and video/audio tracks.	Working extraction pipeline
Phase 4	Accessible Format Generation	Develop the four parallel processing modules: text simplification, text-to-speech audio, caption generation, and clip mapping.	Four functioning output modules
Phase 5	Integration & Testing	Integrate all modules into a single pipeline and dashboard; test with real learners and sample content; fix accessibility and performance issues.	Integrated system, test report
Phase 6	Deployment & Feedback	Deploy the platform, onboard pilot institutions or users, collect usage feedback, and plan iterative improvements.	Live platform, feedback log

7. Functional Requirements

- The system shall allow a user to upload a document in PDF format.
- The system shall extract text and structure from the uploaded document.
- The system shall generate a simplified, easy-read version of the extracted text at a configurable reading level.

- The system shall generate narrated audio from the extracted text with adjustable playback speed.
- The system shall generate synced, timestamped captions for any audio or video content.
- The system shall map key terms to a curated library of sign-language clips.
- The system shall present all generated formats through a single, unified dashboard.

8. Non-Functional Requirements

- Usability: the interface shall be navigable using screen readers and keyboard-only input.
- Performance: processing of a standard document shall complete within an acceptable wait time for a live demo.
- Reliability: generated outputs shall be cached to avoid repeated processing of the same file.
- Scalability: each processing module shall run independently so new formats can be added later.

9. Feature Priority

Requirements are grouped by priority to keep the initial build focused and complete rather than broad.

Priority	Included Requirements
Must Have	Document upload, content extraction, text simplification, text-to-speech audio, caption generation, unified dashboard.
Should Have	Adjustable reading level and playback speed for generated content.
Could Have	Curated sign-language clips for key terms (proof of concept).

10. Libraries and Software to be Used

Category	Library / Software	Purpose
Programming	Python	Backend
Frontend	REACT , VITE	UI
Backend	Python	FastAPI
Database	MySQL	Storage
Captions	SRT GENERATOR	Speech to text
Text-to-Speech	Google TTS	Audio
AI	If API applicable	Regex
Deployment	Streamlit	Web app
Version Control	Git & GitHub	Code management

11. Recommended Software

- Visual Studio Code
- Python 3.10
- MySQL Workbench
- Git
- GitHub
- Postman
- Google Chrome
- FFmpeg

12. Risks & Mitigations

Risk	Mitigation
Full sign-language generation is a complex, unsolved NLP problem.	Scope down to a curated library of key-term clips rather than full sentence generation.
Third-party service limits may be hit during development or demos.	Use free-tier services where possible and cache generated outputs.
Limited development time may cause scope creep.	Get one input type (PDF) working end-to-end before expanding to others.

13. Future Enhancements

- Full avatar-based sign-language generation, beyond curated key-term clips.
- Multi-language support for simplification, audio, and captions.
- Real-time captioning for live lectures.
- Integration with learning management systems such as Google Classroom, Moodle, and Canvas.